DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The CLI To-Do List Manager is a Python-based command-line application designed for task lifecycle management using a local JSON storage pattern. It implements a service-oriented architecture separating CLI dispatch, business logic, and file I/O to enable independent user story implementation and testing.

1.2 Maturity Assessment

The project is structurally sound with a high health index and a clear execution roadmap. While formal acceptance criteria and explicit performance bounds for the JSON storage are missing, the presence of independent test cases for each user story ensures functional verifiability. The project is READY for execution.

1.3 Technical Stack

- Python
- pyproject.toml

1.4 Architectural Constraints

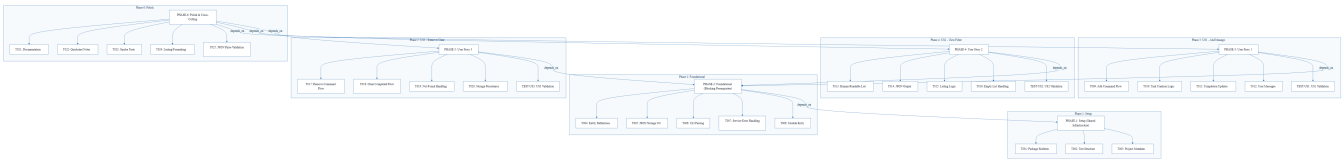
- Sequential ID assignment for task creation.
- Strict JSON serialization for data persistence.
- Storage path resolution targeting `~/.todos.json`.
- Human-readable and JSON output modes for listing.
- Blocking dependency: Foundational Phase (Phase 2) must be complete before any User Story implementation.
- Manual smoke test validation for all core commands (add, list, complete, remove, clear).

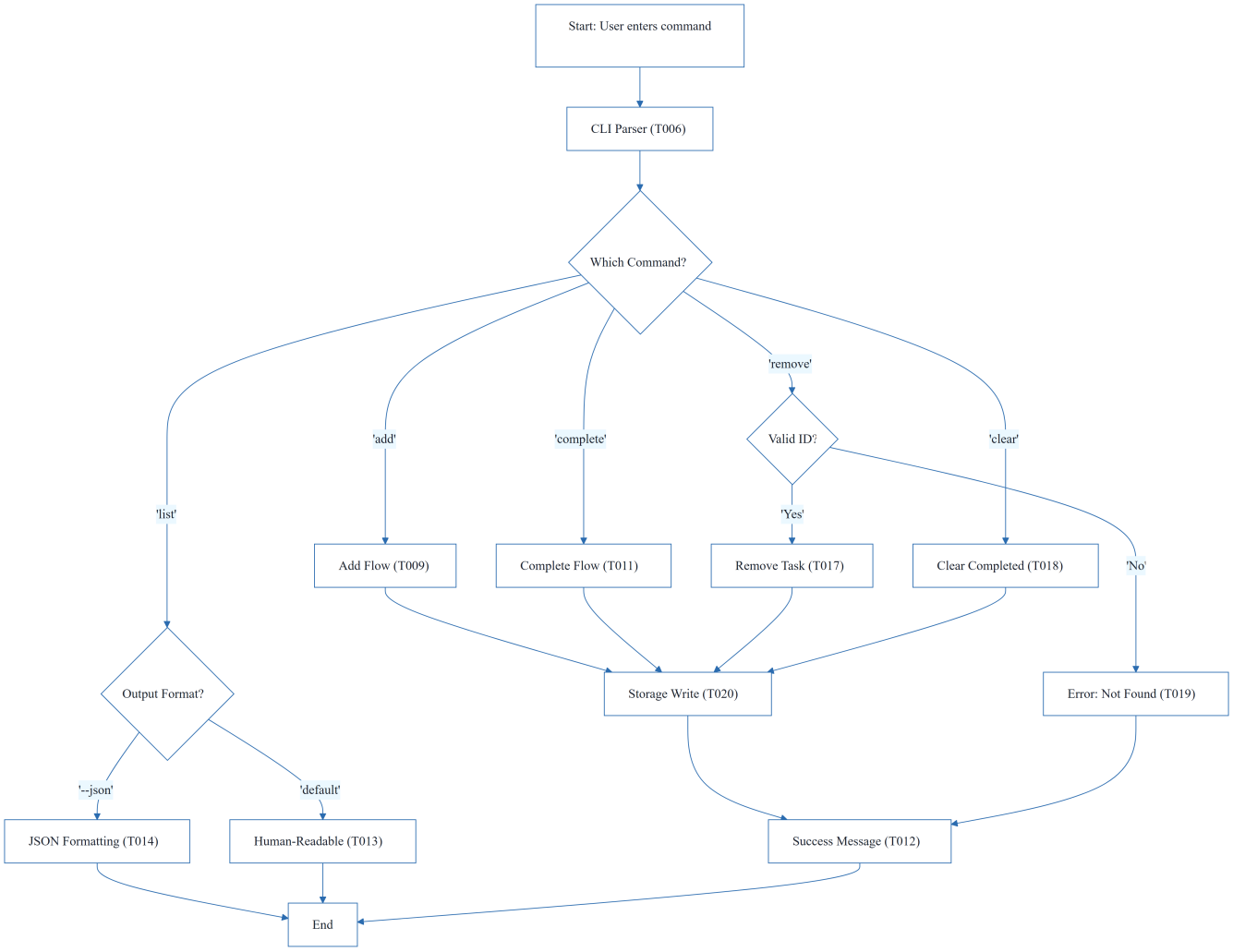
1.5 Critical Dependencies

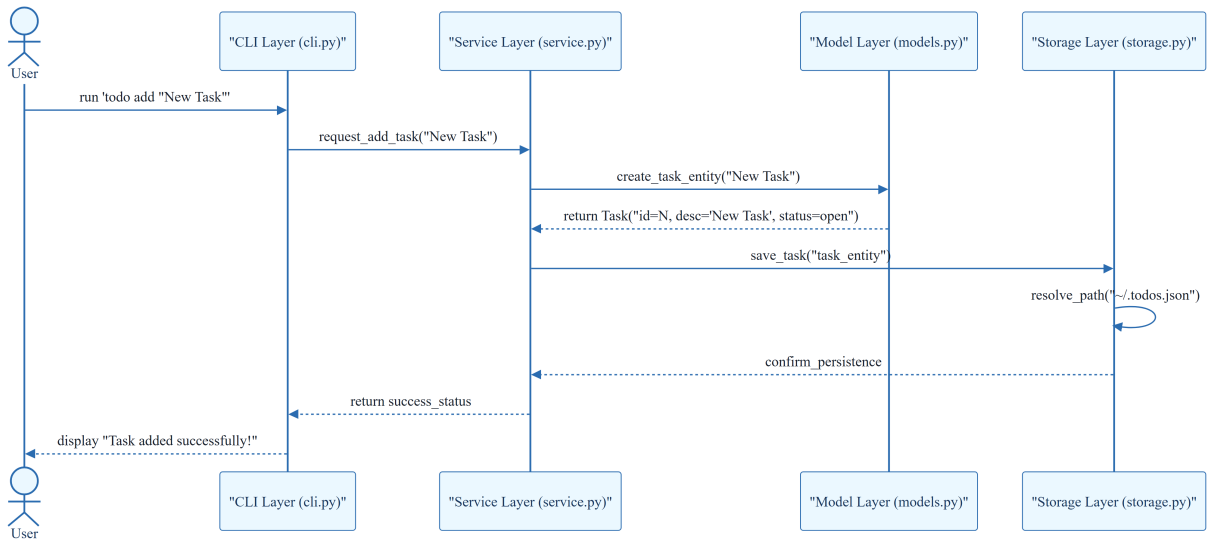
- JSON file I/O for persistence in `~/.todos.json`.
- `pyproject.toml` for CLI entry point and tooling configuration.
- Sequential dependency: Setup $\rightarrow$ Foundational $\rightarrow$ User Stories $\rightarrow$ Polish.
- Internal data integrity: Task entity serialization helpers must precede CLI wiring.

- Referential integrity: Task ID validation for remove and complete operations.

2. Architecture Workflows



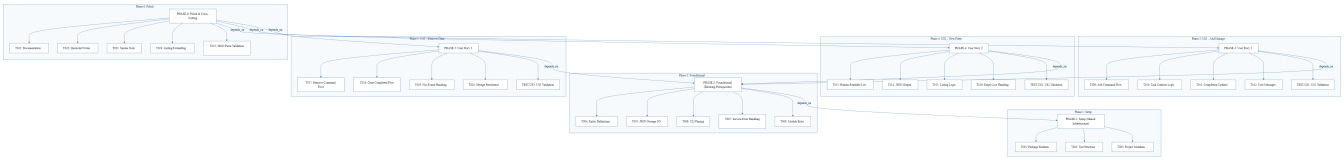




& Visual Diagrams

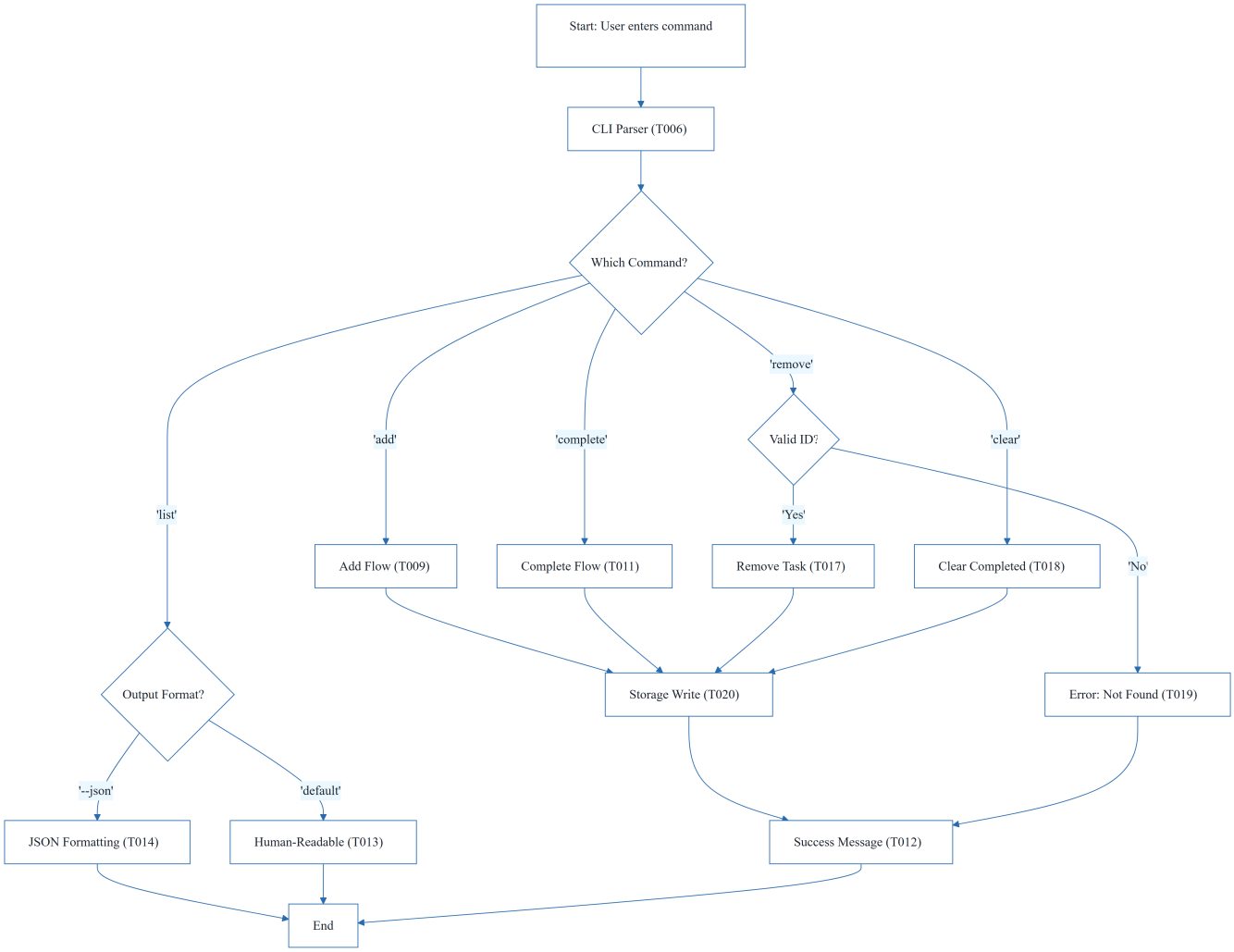
2.1 Project Implementation Roadmap & Traceability

Visualizes the dependency chain between project phases and the specific tasks contained within each phase, ensuring full traceability from setup to polish.



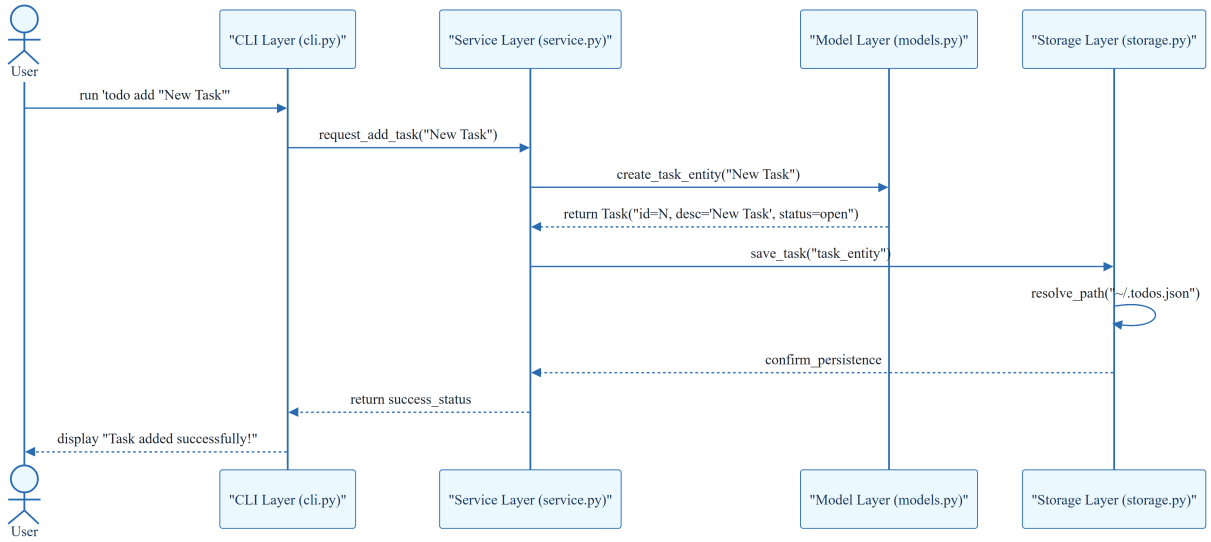
2.2 CLI Command Execution Workflow

Models the business logic flow of the CLI tool, including the dispatch mechanism and decision points for different commands.



2.3 CLI To-Do Manager Sequence Interaction

Illustrates the interaction between the User, CLI layer, Service layer, and Storage layer for a typical 'Add Task' operation.



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Description	Status	Source/Story
T001	Create the Python package skeleton in src/todo_manager/**init**.py, **main**.py, cli.py, models.py, storage.py, and service.py	Completed	PHASE-1
T002	Create the test directory structure in tests/unit/ and tests/integration/	Completed	PHASE-1
T003	Add project metadata and tooling entry points in pyproject.toml	Completed	PHASE-1
T004	Implement task entity definitions and serialization helpers in src/todo_manager/models.py	Completed	PHASE-2
T005	Implement JSON storage path resolution and file I/O helpers in src/todo_manager/storage.py	Completed	PHASE-2
T006	Implement shared command-line parsing and top-level dispatch in src/todo_manager/cli.py	Completed	PHASE-2
T007	Implement shared service-layer error handling and task collection utilities in src/todo_manager/service.py	Completed	PHASE-2
T008	Define module entry behavior for python -m todo*manager in src/todo*manager/**main**.py	Completed	PHASE-2
T009	Implement the `add` command flow in src/todo*manager/cli.py and src/todo*manager/service.py	Completed	US1
T010	Implement task creation, sequential ID assignment, and `created_at` population in src/todo_manager/models.py	Completed	US1

ID	Description	Status	Source/Story
T011	Implement completion updates for existing tasks in src/todo_manager/service.py	Completed	US1
T012	Add user-facing success and error messages for add and complete operations in src/todo_manager/cli.py	Completed	US1
T013	Implement human-readable task listing output in src/todo_manager/cli.py	Completed	US2
T014	Implement `--json` output formatting in src/todo*manager/cli.py using the task serialization helpers in src/todo*manager/models.py	Completed	US2
T015	Add listing logic that preserves task order and status fields in src/todo_manager/service.py	Completed	US2
T016	Handle the empty-list case with a clear message in src/todo_manager/cli.py	Completed	US2
T017	Implement the `remove` command flow in src/todo*manager/cli.py and src/todo*manager/service.py	Pending	US3
T018	Implement the `clear` command flow for removing completed tasks in src/todo_manager/service.py	Pending	US3
T019	Add not-found handling for invalid task IDs in src/todo_manager/cli.py	Pending	US3
T020	Ensure storage writes persist removals and clears safely in src/todo_manager/storage.py	Pending	US3
T021	Document the CLI usage and storage behavior in README.md	Pending	PHASE-6
T022	Add quickstart verification notes and examples in specs/001-cli-todo-manager/quickstart.md	Pending	PHASE-6

ID	Description	Status	Source/Story
T023	Run a manual smoke test of add, list, complete, remove, and clear against ~/.todos.json	Pending	PHASE-6
T024	Verify linting and formatting expectations for the source files in src/todo_manager/	Pending	PHASE-6
T025	Confirm the generated JSON output remains parseable for the todo list --json path	Pending	PHASE-6
TEST-US1	Run `todo add "Task description"` and `todo complete` , then confirm the stored task list reflects the new task and its completion status.	N/A	US1
TEST-US2	Run `todo list` and `todo list --json` and confirm the output is readable or parseable JSON while showing the full task collection.	N/A	US2
TEST-US3	Run `todo remove` and `todo clear`, then confirm the targeted task or completed tasks are removed while other tasks remain intact.	N/A	US3

3.2 Security Rules

- No explicit security rules defined in source.
- Recommended: Implement input sanitization for task descriptions to prevent injection or formatting issues in JSON.

3.3 Data Models

- **Task Entity**:
- `id`: Sequential alphanumeric token (T010).
- `description`: String.
- `status`: Open/Completed.
- `created_at`: Timestamp populated during instantiation (T010).
- **Persistence**: JSON file located at `~/.todos.json` (T005).

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Missing Section	Priority	Remediation Advice
Acceptance Criteria	MEDIUM	While 'Independent Tests' are provided, formal acceptance criteria for each task would improve validation.
Security & Performance Constraints	LOW	The project is a simple CLI tool, but constraints on JSON file size or input sanitization could be added.
Open Questions & Uncertainties	LOW	No open questions were listed in the source document.

4.2 Remediation & Workflow

1. **Validation**: Integrate formal acceptance criteria into the "Polish" phase (Phase 6).
2. **Performance**: Define a maximum JSON file size limit to prevent CLI degradation.
3. **Security**: Add a validation layer in `service.py` to sanitize user input before it reaches `models.py`.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Checkpoint	TECHNICAL_STACK	PHASE-2	A synchronization gate ensuring all blocking infrastructure is verified before parallel feature development commences.
Foundational	TECHNICAL_STACK	PHASE-2	The core shared infrastructure layer containing serialization and I/O helpers that block all subsequent user story implementations.
Goal	BUSINESS_DOMAIN	PHASE-3	The primary functional objective a user must achieve within a specific feature set.
ID	TECHNICAL_STACK	T010	A sequential alphanumeric token assigned to each entry to ensure unique referenceability.

Term	Category	Context Anchor	Project Definition
JSON	TECHNICAL_STACK	T005	The lightweight data-interchange format used for persistent storage in the home directory.
MVP	BUSINESS_DOMAIN	PHASE-3	The minimum viable product consisting of the first priority user story for basic task creation and completion.
Organization	BUSINESS_DOMAIN	Tasks: CLI To-Do List Manager	The structural grouping of implementation units by user story to ensure independent testability.
Prerequisites	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The set of mandatory design documents including plan, spec, and data-model required before coding.
README	TECHNICAL_STACK	T021	The primary documentation file detailing command-line usage and storage behavior.
Setup	TECHNICAL_STACK	PHASE-1	The initial phase involving package skeleton creation and tooling configuration in the project metadata file.
T016	TECHNICAL_STACK	T016	The specific implementation unit responsible for providing a clear message when the task collection is empty.
Tests	TECHNICAL_STACK	T002	The directory structure for unit and integration verification, validated via quickstart smoke tests.
population in	TECHNICAL_STACK	T010	The process of assigning a timestamp to the creation field during entity instantiation.
todo clear	BUSINESS_DOMAIN	T018	The operation that removes all entries marked as finished from the persistent store.
todo list	BUSINESS_DOMAIN	T013	The operation that retrieves and displays all entries in either human-readable or machine-parseable formats.

Term	Category	Context Anchor	Project Definition
■■ CRITICAL	TECHNICAL_STACK	PHASE-2	A high-severity constraint indicating that no feature work can proceed until the current phase is fully completed.